

Reading and writing text files

CSC103 Programming Fundamentals / Lecture 29

Dr. Muhammad Farhan

Associate Professor of Computer Science
Department of Computer Science
COMSATS University Islamabad
Sahiwal Campus

Fall 2026

The problem for this lecture

Read a file of integer marks and reject the first malformed token. Print No data for an empty file, otherwise the average.

Create a dedicated UTF-8 fixture containing 60 and 80 for the demonstration.

Learning objectives

- Write text with `PrintWriter`
- Read data with `Scanner`
- Use `try-with-resources` and explicit encoding

CLO-3

`PrintWriter` output

`Scanner` file input

`Try-with-resources`

A file-processing contract

PrintWriter output

- PrintWriter writes formatted text to a destination.
- println writes a line and printf supports a format string.
- Opening the demonstrated writer replaces existing file contents.

Example

```
try (java.io.PrintWriter out =  
    new java.io.PrintWriter("demo.txt", "UTF-8")) {  
    out.println(60);  
    out.println(80);  
}
```

Scanner file input

- Scanner can parse a text file into tokens.
- hasNextInt checks whether the next token matches an integer.
- Reading must advance even when a token is invalid.

Example

```
try (java.util.Scanner in =  
    new java.util.Scanner(new java.io.File("demo.txt"), "UTF-8")) {  
    while (in.hasNextInt()) {  
        System.out.println(in.nextInt());  
    }  
}
```

Try-with-resources

- Declare resources in the try header.
- Java closes them when the block exits.
- Multiple resources close in reverse declaration order.

Example

```
try (Resource r = openResource()) {  
    // use r  
}  
// r closes on normal or exceptional exit
```

A file-processing contract

- Define the encoding and record format.
- Choose what malformed input means: reject it or report and skip it.
- Treat empty files and missing files as separate cases.

Example

Contract `for this` example:

UTF-8 integer tokens

Malformed token: reject the file

Empty file: sum is zero

A reader must consume or reject every token

- A loop controlled by hasNext continues while a token exists.
- If the token is not an integer, the program must consume it or end with a failure.
- Doing neither leaves the same token waiting forever.

Example

```
while (in.hasNext()) {  
    if (!in.hasNextInt()) throw ...;  
    int mark = in.nextInt();  
}
```


The empty case differs from malformed data

- An empty file has no marks and therefore no mean.
- A malformed token violates the declared integer format.
- Handling these as different outcomes gives a useful explanation to the caller.

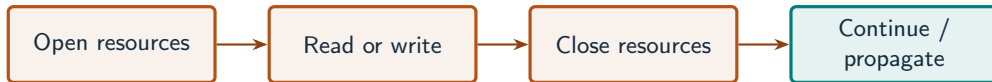
Example

Empty file: No data

"60 80": average 70.0

"60 bad": reject malformed token

Resources have a controlled lifetime



What to notice

Try-with-resources attempts closure when the body exits normally or by exception.

Key distinctions

File contents	Interpretation	Required outcome
60 80	Two valid marks	Average 70.0
Empty	Zero records	No data
60 bad	Malformed token	Reject with token context
60 101	Out-of-range mark	Reject range violation

These distinctions determine which operation or control structure fits the problem.

First example: execution

```
String path = "lecture29-demo.txt";
try (java.io.PrintWriter out = new java.io.PrintWriter(path, "UTF-8")) {
    out.println(60); out.println(80);
}
int sum = 0;
try (java.util.Scanner in = new java.util.Scanner(new java.io.File(path), "UTF-8"))
{
    while (in.hasNextInt()) sum += in.nextInt();
}
System.out.println(sum);
```

First example: result

140	Try-with-resources
The writer closes before the reader opens.	A file-processing contract
The dedicated demonstration file contains 60 and 80.	

Guided practice

Read a file of integer marks and reject the first malformed token. Print No data for an empty file, otherwise the average.

Attempt the solution before continuing.
Use the definitions and examples from this lecture.

Solution strategy

- Create a dedicated UTF-8 fixture containing 60 and 80 for the demonstration.
- Read tokens inside try-with-resources, rejecting malformed or out-of-range values.
- Compute a mean only when count is positive, and check for a recorded Scanner I/O failure.

The next program uses fixed inputs so its result can be reproduced.

Complete solution (1/3)

```
public class Solution29 {  
    public static void main(String[] args) throws Exception {  
        String path = "expanded29-demo.txt";  
        try (java.io.PrintWriter out = new java.io.PrintWriter(path, "UTF-8")) {  
            out.println("60 80");  
        }  
        reportAverage(path);  
    }  
    static void reportAverage(String path) throws Exception {  
        long sum = 0;
```


Complete solution (2/3)

```
int count = 0;
try (java.util.Scanner in = new java.util.Scanner(
    new java.io.File(path), "UTF-8")) {
    while (in.hasNext()) {
        if (!in.hasNextInt()) {
            throw new IllegalArgumentException("Bad token: " + in.next());
        }
        int mark = in.nextInt();
        if (mark < 0 || mark > 100) {
            throw new IllegalArgumentException("Mark outside 0..100");
        }
    }
}
```

Complete solution (3/3)

```
    }  
    sum += mark;  
    count++;  
}  
if (in.ioException() != null) throw in.ioException();  
}  
if (count == 0) System.out.println("No data");  
else System.out.println((double) sum / count);  
}  
}
```

Execution trace

Next token	Action	Sum	Count
60	Parse and validate	60	1
80	Parse and validate	140	2
End of file	Stop reading	140	2
Report	140 / 2.0	70.0 mean	2

Follow the changing state in execution order.

Solution result

70.0

Why this result follows
Compute a mean only when count is positive, and check for a recorded Scanner I/O failure.

A common incorrect approach

```
while (in.hasNext()) {  
    if (in.hasNextInt()) sum += in.nextInt();  
}
```

Example

A non-integer token is never consumed, so the loop repeats forever. Consume and report it, or `throw` a clear exception.

Boundary and failure checks

Test an empty file, a malformed token and a mark outside 0..100.

Loop while `in.hasNext()`. If `!in.hasNextInt()`, throw an exception naming `in.next()`. Otherwise accumulate `nextInt` and count. Divide only when `count > 0`. Close via `try-with-resources`.

Check your understanding

Why close the writer before reading its output? What happens to existing data when this writer opens?

Write a prediction and explain the rule that supports it.

Answer and explanation

Closing flushes buffered text and releases the resource.
This constructor truncates existing file contents.

A non-integer token is never consumed, so the loop repeats forever. Consume and report it, or throw a clear exception.

Compare cases and predict outcomes

File content	Required action	Reason
60 80	Report mean 70.0	Two valid marks
60 bad	Reject input	Do not silently skip malformed data
Empty	Report No data	No defined mean

Discuss

Explain each expected result before running the example. Which case would reveal a wrong assumption?

Apply the idea

Independent practice

Write a short try-with-resources fragment that scans all integer tokens from marks.txt. Reject a non-integer token instead of ending silently.

1. Work through the input by hand.
2. Write or correct the program.
3. Justify the expected result.

Solution and reasoning

```
try (java.util.Scanner in = new java.util.Scanner(  
    new java.io.File("marks.txt"), "UTF-8")) {  
    while (in.hasNext()) {  
        if (!in.hasNextInt())  
            throw new IllegalArgumentException("Bad token: " + in.next());  
        System.out.println(in.nextInt());  
    }  
}
```

- hasNext() asks whether any token remains; hasNextInt() checks the expected type.
- The fragment must be inside a method that catches or declares the file-opening exception.
- For a robust complete reader, also check Scanner.ioException() after reading.

Lecture summary

- text with `PrintWriter`
- data with `Scanner`
- `try-with-resources` and explicit encoding
- Create a dedicated UTF-8 fixture containing 60 and 80 for the demonstration.
- Read tokens inside `try-with-resources`, rejecting malformed or out-of-range values.
- Compute a mean only when count is positive, and check for a recorded `Scanner` I/O failure.

After-class practice

Write three marks to a practice file and read them back.
Test an empty file, malformed text and a missing input path.

Syllabus reading
Liang Ch 12
Next lecture: Test harnesses